International Institute of Information Technology
Bangalore

PL project Report

Team Members:

Abhinav Kumar (IMT2022079)
Kushal Suvan Jenamani (IMT2022057)

September 3, 2026

Introduction

Metaprogramming is a computer programming technique in which computer programs have the ability to treat other programs as their data. It means that a program can be designed to read, generate, analyse, or transform other programs, and even modify itself, while running.

Uses of Metaprogramming

- **Code Reusability and Abstraction :** Metaprogramming can help reduce code duplication by allowing you to write more generic and reusable code. For example, you can write a single piece of code that generates various classes or functions based on a set of parameters or a template.
- **Move code to compile time :** Metaprogramming can be used to move computations from runtime to compile time, to generate code using compile time computations, and to enable self-modifying code. The ability of a programming language to be its own metalanguage allows reflective programming, and is termed reflection.
- **Dynamic Behavior :** It enables programs to adapt their behavior at runtime. For example, you can use metaprogramming to implement custom serialization or deserialization logic, which can be useful for handling different data formats or adapting to changes in data structures.
- **Reducing Boilerplate Code :** Metaprogramming can automate the creation of boilerplate code, such as getters and setters, or implement design patterns like the Factory pattern. This reduces the amount of repetitive code you need to write and maintain.
- **Domain-Specific Languages (DSLs) :** It allows you to create DSLs, which are specialized languages tailored to a specific problem domain. For instance, Ruby on Rails uses metaprogramming to provide a powerful DSL for building web applications.
- **Code Generation :** In some cases, metaprogramming is used to generate code based on certain rules or configurations. This can be especially useful in scenarios where the code needs to be generated dynamically, like in template engines or for integrating with different APIs.
- **Dynamic code generation :** Dynamic code generation is a metaprogramming technique where a program creates, modifies, or executes new source code while it is running. Instead of relying only on pre-written logic, the software uses templates or strings to build and run code on the fly.
- **Reflection and Introspection :** Metaprogramming allows for reflection and introspection, meaning the ability to inspect and modify the structure and behavior of programs at runtime. This can be useful for debugging, logging, and other dynamic operations.

There are many other use cases in which metaprogramming can levraged to our advantage.

MetaProgramming and Types

> MetaProgramming is the system that allows a program to generate programs.

We always have treated programs as black-boxes, that take in some or no data, and output some or no data. This intuitively suggests that any implementation of MetaProgramming will blur the lines between these black-boxes (a.k.a. programs) and data.

As we have already seen, this is the case in all languages we have discussed till now- metaprogramming often deals with programs and data as the same first class objects.

A bit more rigorous definition

The Universal Turing Machine is the standard way we talk about such setups.

A program is a encoding of Turing Machine M. A UTM reads from this tape, and accordingly manipulated encoding on another tape a.k.a. the data.

Von Neumann architecture extends this slightly, by conflating that read-only program tape, and the read-write data tape. So, now program and data live in the same tape.

This brings us to the second alternative and equivalent definition for MetaProgramming.

> MetaProgramming is a system where code and data live in the same space and are treated as the same.

Formal Logic: A primer on syntax and semantics

We know about the different stages of compilations of programs- the parser extracts the terms, the sytax analyzer enforces the grammar of the language while semantic analyzer checks for the validity of the program from a "meaningful" persepective.

This pipeline's existence can be traced back directly to formal logic.

Syntax is rigorously constructed as follows

$$\mathcal{L} = \{\mathcal{C}, \mathcal{F}, \mathcal{P}\}$$

This is what a language consists of. First is a set of constants $\mathcal{C}$. It can contain anything that we intend to use as symbols. The second thing is the set of k-ary functions $\mathcal{F}$. Thus,

$$\mathcal{F} \coloneqq \{f^k : \mathcal{C}^k \to \mathcal{C}\}$$

And finally, the predicates are defined as follows

$$\mathcal{P} \coloneqq \{p^m : \mathcal{C}^m \to \{0, 1\}\}$$

The programming contexts do not prove stuff, and thus the use of $\mathcal{P}$ turn out to be limited. But $\mathcal{C}$ and $\mathcal{F}$ are imported as it is, in the form of "terms".

Next, we define inductively what terms are.

$$\mathcal{T} \coloneqq c \in \mathcal{C} | x \in \mathcal{V} | f^k(t_1, \ldots, t_k), \text{where } t_i \in \mathcal{T}$$

Here, $\mathcal{V}$ is the set of variables.

If you can see, this is exactly what a parser gives us as tokens. Every constant like $1, 3.14$, `true` ... falls under $\mathcal{C}$. Every token like + falls under $\mathcal{F}$, f^2 to be precise as it is a 2-ary function.

Given these terms, if we endow it with Context-Free Grammar, we will get a validated AST.

What remains is the semantics.

This is a little subtle. Till this point, we have said nothing about what + is. If you pay attention, the program can always compile + to a `SUB` instruction at assembly level. The point being, this is just a syntactic check, ensuring if we have what we call *well-formed sentences*.

Type Theory and Semantics

Now that we have shown what it takes to rigorously establish semantics of a language, we add the thin layer of explanantion of how Type Theory achieves that.

This will br our next work, where we rigorously show why type systems are in continuous friction with MetaProgramming, how it is fundamentally impossible, relating it back to the foundational results in mathematical logic like Gödel's Incompletenes Theorem and computability theory like the Halting Problem.

Lisp

Lisp stands for List Processor. The reason for which will become evident.

We will start by positing yet another equivalent definition for MetaProgramming, that hits home for Lisp and the languages ahead.

> MetaProgramming is the ability to program directly the Abstract Syntax Trees.

Take a moment for this to sink in. Because data and program are treated the same, and as proven earlier, semantics are the only thing we cannot code in full completeness, what it leaves us is a "partially compiled program". It just a program that has passed through the lexer and syntax analyzer. This is equivalent to writing, manipulating, processing the AST manually, as data.

Now, Lisp makes this interesting for the following reason- it has flattened the Abstract Syntax "tree" in Abstract Syntax "lists". The reason is that we write binary functions like + in Lisp in pre-order format. So $2 + 3$ is written as $+2\,3$. Now, where there was supposed to be a + node in the AST, with left and right children as 2 and 3, we just have a linked list of $\boxed{+} \to \boxed{2} \to \boxed{3}$.

After we have written our AST, which is a list in this case, we pass it to the `eval` method, which actually executes it. Thus the name- List Processor.

Code as Data - Basic List Manipulation

```lisp
;; This is just a list - not executed
(setq code-as-data '(+ 2 3))

;; Inspect the AST (which is just a list)
(println "First element: " (first code-as-data))    ; => +
(println "Second element: " (nth 1 code-as-data))   ; => 2

;; Now execute the AST
(println "Result: " (eval code-as-data))            ; => 5
```

```
;; Modify the AST before execution
(setq modified-code (cons '* (rest code-as-data))) ; (+ 2 3) -> (* 2 3)
(println "Modified result: " (eval modified-code)) ; => 6
```

The quote ' prevents evaluation, giving us the raw AST as a list. We manipulate it with `first`, `rest`, `cons` - standard list operations. Then `eval` executes it.

Building Code Programmatically

```
;; Function that generates code based on operator
(define (make-operation op a b)
  (list op a b))  ; Constructs AST manually

;; Build different ASTs
(setq add-code (make-operation '+ 10 20))
(setq mul-code (make-operation '* 10 20))

;; Execute them
(println (eval add-code))  ; => 30
(println (eval mul-code))  ; => 200
```

We are constructing ASTs at runtime using list operations, then executing them. The AST is just `(operator arg1 arg2)`.

Macros - Compile-Time AST Transformation

```
;; Macro that transforms code before execution
(define-macro (unless condition body)
  (list 'if (list 'not condition) body))

;; Usage
(unless (> 5 10)
  (println "5 is not greater than 10"))

;; What the macro does:
;; Input AST:  (unless (> 5 10) (println "..."))
;; Output AST: (if (not (> 5 10)) (println "..."))
;; The transformed AST is then evaluated
```

Macros receive the unevaluated AST as input, transform it, and return a new AST that gets evaluated. You are literally rewriting the syntax tree.

Dynamic Function Generation

```
;; Generate multiple similar functions programmatically
(define (make-adder n)
```

```
  (lambda (x) (+ x n)))  ; Returns a function (which is also data!)

(setq add5 (make-adder 5))
(setq add10 (make-adder 10))

(println (add5 3))   ; => 8
(println (add10 3))  ; => 13

;; Generate functions from AST templates
(define (make-binary-op op)
  (eval (list 'lambda '(x y) (list op 'x 'y))))

(setq my-multiply (make-binary-op '*))
(println (my-multiply 4 5))  ; => 20
```

We are constructing the AST `(lambda (x y) (* x y))` as a list, then evaluating it to create a function. The function definition itself is data.

Ruby

Ruby, being an object-oriented and dynamic programming language, has built-in support for metaprogramming techniques. Metaprogramming in Ruby can be used in a variety of ways:

- Dynamically defining methods and classes
- Modifying existing classes and methods
- Inspecting and manipulating code objects

Metaprogramming makes Ruby code more powerful, flexible, and expressive. It can also create new features and functionality for the Ruby language itself.

```
class Order
  def initialize(order_no)
    @order_no = order_no
  end

  def initated()
    puts "initated #{@order_no}"
  end

  def dispatched()
    puts "dispatched #{@order_no}"
  end

  def out_for_delivery()
    puts "out_for_delivery #{@order_no}"
  end
```

```ruby
  def delivered()
    puts "delivered #{@order_no}"
  end

  attr_accessor(:order_no)

  def take_action(action)
    return if !self.respond_to?(action)

    self.send(action)
  end
end

def main
  order_123 = Order.new(123)
  order_123.take_action("initated")
  order_123.order_no = 789
  order_123.take_action("dispatched")
  order_123.take_action("out_for_delivery")
  order_123.take_action("delivered")
  order_123.take_action("bruh")
end

if __FILE__ == $0
  _ = main()
end
```

The `send` method in Ruby is used to dynamically call methods on objects. It takes two arguments: the name of the method to call and any arguments that should be passed to the method. The method name can be passed as a string or a symbol. This method can also call private methods.

The `responds_to?` method in ruby returns `true` if object responds to the given method. Private and protected methods are included in the search only if the optional second parameter evaluates to `true`.

```ruby
class OrderStatus
  def initialize(order_no)
    @order_no = order_no
    @status = "initiated"
  end
  STATUSES = ["initiated","dispatched","out_for_delivery","delivered","cancelled"]
  STATUSES.each do |status|
    define_method("is_#{status}?") do
      @status == status
    end
  end
  STATUSES.each do |status|
```

```
    define_method("to_#{status}") do
      @status = status
    end
  end
  attr_accessor(:order_no,:status)
end

def main
  order_123 = OrderStatus.new(123)
  pp(order_123.is_initiated?())
  order_123.to_dispatched()
  pp(order_123.is_dispatched?())
  order_123.to_cancelled()
  pp(order_123.is_cancelled?())
end

if __FILE__ == $0
  _ = main()
end
```

The `define_method` method allows developers to define new methods at runtime. It takes two arguments: the name of the new method and a block of code that defines the behavior of the method

The `attr_accessor` method allows us to define getter and setter of attributes of class by taking in symbol or string of variable.

```
class Square
  def initialize()

  end
  def method_missing(method_name, *args, &block)
    method_n = method_name.to_s
    if method_n.start_with?("area_")
      length = method_n.gsub("area_", "").to_i
      return length*length
    elsif method_n.start_with?("perimeter_")
      length = method_n.gsub("perimeter_", "").to_i
      return 4*length
    else
        super
    end
  end
end


def main
  p = Square.new()
```

```
  puts p.area_50000
  puts p.perimeter_50000
end

if __FILE__ == $0
  _ = main
end
```

The `method_missing` is a method that ruby gives you access inside of your objects a way to handle situations when you call a method that doesn't exist.

```
class String
  def reverse
    super.split("").reverse.join("")
  end
end

puts "hello, world".reverse
```

Monkey patching in Ruby is a metaprogramming technique that allows you to modify the behaviour of existing classes and modules at runtime. It can be done by reopening the class or module and adding or overriding methods.

```
class Square
end

Square.class_eval do
    def area(length)
        puts "Area = #{length * length}"
    end
end

Square.new.area(5)
```

The `class_eval` method is used to evaluate a block of code within the context of a class, allowing you to dynamically add or modify methods on that class at runtime.

```
class Square
end

square = Square.new

square.instance_eval do
    def area(length)
        puts "Area = #{length * length}"
    end
end
```

```
square.area(5)
```

The `instance_eval` method is similar to `class_eval`, but it evaluates code within the context of an object rather than the class itself. This can be useful for dynamically adding or modifying methods on a particular object at runtime.

Reference Links

- **Article on Meta programming**
  https://alpiopio.medium.com/meta-programming-2b35b5619b63
- **Metaprogramming in Ruby**
  https://www.shakacode.com/blog/metaprogramming-in-ruby/
- **Metaprogramming in Ruby - Scaler**
  https://www.scaler.com/topics/metaprogramming-in-ruby/

Book References

- **Formal Logic:**
  Herbert B. Enderton, *A Mathematical Introduction to Logic*, Academic Press, 2001.
- **Lisp:**
  Paul Graham, *On Lisp: Advanced Techniques for Common Lisp*, Prentice Hall, 1993.
  (Available free: http://www.paulgraham.com/onlisp.html)
- **Type Theory:**
  Benjamin C. Pierce, *Types and Programming Languages*, MIT Press, 2002.